



INFORME DEL TRABAJO PRACTICO

I. PORTADA

Tema:	SW-AyLP-APE-06: Arreglos y Listas
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Primero Software “B”
Alumnos participantes:	Sánchez Lema Isaac Adrián
	
Asignatura:	ALGORITMOS Y LÓGICA DE PROGRAMACIÓN
Docente:	Ing. José Caiza, Mg.

SW-AYLP-APE-06: ARREGLOS Y LISTAS

II. INFORME DEL TRABAJO PRACTICO

1.1 Objetivos

General:

- Desarrollar y consolidar las habilidades en lógica de programación y el manejo de estructuras de datos lineales (estáticas y dinámicas) mediante la resolución de problemas prácticos en los lenguajes Java y C++, fundamentando las bases técnicas necesarias para el diseño y construcción de herramientas tecnológicas orientadas a la enseñanza y fortalecimiento del pensamiento computacional.

Específicos:

- **Analizar y diseñar** soluciones algorítmicas mediante el uso de colecciones dinámicas (ArrayList / vector) y estructuras de nodos enlazados (LinkedList / list) para optimizar el almacenamiento, búsqueda y manipulación de información.
- **Implementar** mecanismos estrictos de validación de datos de entrada y control de excepciones en los flujos lógicos, garantizando la robustez, estabilidad y prevención de fallos del software en tiempo de ejecución.
- **Evaluar** de forma comparativa el rendimiento, comportamiento estructural y la gestión de memoria de los arreglos estáticos frente a las listas dinámicas bajo la filosofía de estructuras avanzadas (como la lógica FIFO).

1.2 Modalidad

Presencial.

1.3 Tiempo de duración

Presenciales; 6.

No presenciales; 0.

1.4 Instrucciones

- La tarea se debe realizar de forma individual.



- Lea detenidamente cada uno de los ejercicios planteados en el aula virtual y desarrolle lo solicitado.
- Codifique en Java cada uno de los ejercicios planteados.
- Al finalizar, comprimir y subir al aula virtual el proyecto desarrollado, adicionalmente realizar capturas de cada una de las clases y métodos de los ejercicios.
- Las capturas se subirán en un archivo pdf en el formato de Guías APE.

1.5 Listado de equipos, materiales y recursos

Inteligencia artificial, TAC Inteligencia artificial, TAC Inteligencia artificial, TAC

- Computador o laptop.
- Plataforma educativa de la UTA.
- Internet para consultas. - Papel - Material de escritorio

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☒ Simuladores y laboratorios virtuales
- ☒ Aplicaciones educativas
- ☒ Recursos audiovisuales
- ☒ Gamificación
- ☒ Inteligencia Artificial
- Otros (Especifique): _____

1.6 Actividades para desarrollar

Desarrollar la guía APE en base al formato y las instrucciones brindadas

1.7 Resultados obtenidos

- **Resolución técnica y estructurada** de 10 problemas algorítmicos complejos, cada uno respaldado por su análisis del problema, tablas conceptuales de equivalencia de variables (Java/C++) y el diseño lógico en pseudocódigo.
- **Codificación funcional y documentada** en entornos multiplataforma para los lenguajes Java y C++, logrando una asimilación práctica de la sintaxis y de las librerías nativas de colecciones (java.util y la Standard Template Library - STL).
- **Mitigación total de excepciones comunes** (como desbordamientos de memoria u omisiones por búfer sucio) a través de la incorporación de validaciones de rangos numéricos y restricciones para cadenas de texto vacías.
- **Evidencia empírica del rendimiento** institucional de las estructuras mediante pruebas de inserción masiva, constatando la flexibilidad de redimensionamiento automático que ofrecen los contenedores dinámicos.

1.8 Habilidades blandas empleadas en la práctica

- ☒ Liderazgo
- ☒ Trabajo en Equipo
- ☒ Comunicación asertiva
- ☒ La empatía



- ☒ Pensamiento crítico
- ☒ Flexibilidad
- ☒ La resolución de conflictos
- ☒ Adaptabilidad
- ☒ Responsabilidad

1.9 Conclusiones

- La correcta selección de estructuras de datos es un factor crítico en la ingeniería de software. Mientras que los arreglos tradicionales imponen restricciones severas debido a su tamaño fijo e inmutable, las colecciones dinámicas como ArrayList y vector proporcionan una flexibilidad superior que optimiza el uso de la memoria según las necesidades cambiantes del entorno.
- La implementación sistemática de validaciones robustas en la captura de datos reduce drásticamente la vulnerabilidad del software ante errores humanos o ingresos inesperados, lo que asegura que el flujo lógico mantenga su estabilidad operativa y cumpla de manera estricta con el propósito para el cual fue diseñado.
- La ejecución y comprensión profunda de esta actividad práctica constituye un pilar metodológico e instrumental indispensable para el desarrollo de un aplicativo educativo.
- Dominar la lógica detrás de la manipulación de listas, colas y validaciones permite estructurar con rigurosidad el núcleo de una plataforma de enseñanza; esto es clave para diseñar módulos interactivos, desafíos de código controlados y sistemas de retroalimentación inmediata que ayuden a los estudiantes de primer semestre a asimilar conceptos abstractos y superar las curvas de aprendizaje iniciales de la carrera.

1.10 Recomendaciones

- Profundizar en el estudio de la complejidad algorítmica (notación Big O) aplicada a estructuras de datos dinámicas, con el fin de prever el comportamiento y la velocidad de respuesta del software cuando se manejen volúmenes masivos de información.
- Adoptar el hábito de documentar y comentar el código fuente bajo estándares internacionales de desarrollo (como Javadoc o esquemas de documentación estructurada en C++), facilitando el mantenimiento, la escalabilidad y el trabajo colaborativo en futuros proyectos de software.
- Integrar la lógica de resolución, los casos de prueba y las validaciones estrictas analizadas en estos ejercicios dentro de la arquitectura funcional del aplicativo educativo en desarrollo.
- Se recomienda transformar estos problemas prácticos en plantillas o niveles interactivos ramificados dentro de la aplicación, sirviendo como una guía paso a paso para que los estudiantes de primer semestre ejerciten su lógica, comprendan las equivalencias de lenguajes y aprendan a escribir código limpio y tolerante a fallos desde el inicio de su formación profesional.



1.11 Referencias bibliográficas

[1] isma1402uta-boop, "SW-AyLP-APE-06-Arreglos-y-Listas: Ejercicios 1-3 APE", GitHub, 2026. [En línea]. Disponible en: <https://github.com/isma1402uta-boop/SW-AyLP-APE-06-Arreglos-y-Listas>



GUÍA DE RESOLUCIÓN: EJERCICIOS DE ESTRUCTURAS DE DATOS (JAVA Y C++)

Ejercicio 1: ArrayList I - Registro de Estudiantes

Enunciado: Crear un ArrayList con 5 nombres de estudiantes.

1. ANÁLISIS DEL PROBLEMA

- **Entrada:** 5 nombres de estudiantes (cadenas de texto).
- **Proceso:** Almacenar de forma dinámica los nombres en una lista. Validar que las entradas no estén vacías.
- **Salida:** Mostrar la lista de los estudiantes registrados.

2. DEFINICIÓN DE VARIABLES Y EQUIVALENCIAS

Componente del Programa	Tipo de Dato (Java)	Tipo de Dato (C++)	Descripción
listaEstudiantes	ArrayList<String>	vector<string>	Lista dinámica para almacenar los nombres.
nombre	String	string	Variable temporal para leer cada nombre.
i	int	int	Contador para el ciclo de lectura.

3. ALGORITMO LÓGICO

- Inicializar la lista dinámica vacía.
- Repetir 5 veces (desde $i = 1$ hasta 5):
- Leer una cadena de texto (nombre).
- Validar que la cadena no contenga solo espacios en blanco. Si no es válida, volver a solicitar.
- Insertar el nombre al final de la lista.
- Recorrer la lista desde el inicio hasta el final.
- Mostrar en pantalla cada uno de los nombres almacenados.

4. CÓDIGO EN PSEINT

Algoritmo RegistroEstudiantes

Dimension listaEstudiantes[5] // Simulación con arreglo estático en PSeInt

Definir nombre Como Cadena

Definir i Como Entero

Para i <- 1 Hasta 5 Con Paso 1 Hacer

Repetir

 Escribir "Ingrese el nombre del estudiante ", i, ":"

 Leer nombre

 Si Longitud(nombre) = 0 Entonces

 Escribir "Error: El nombre no puede estar vacío."

 FinSi

 Hasta Que Longitud(nombre) > 0

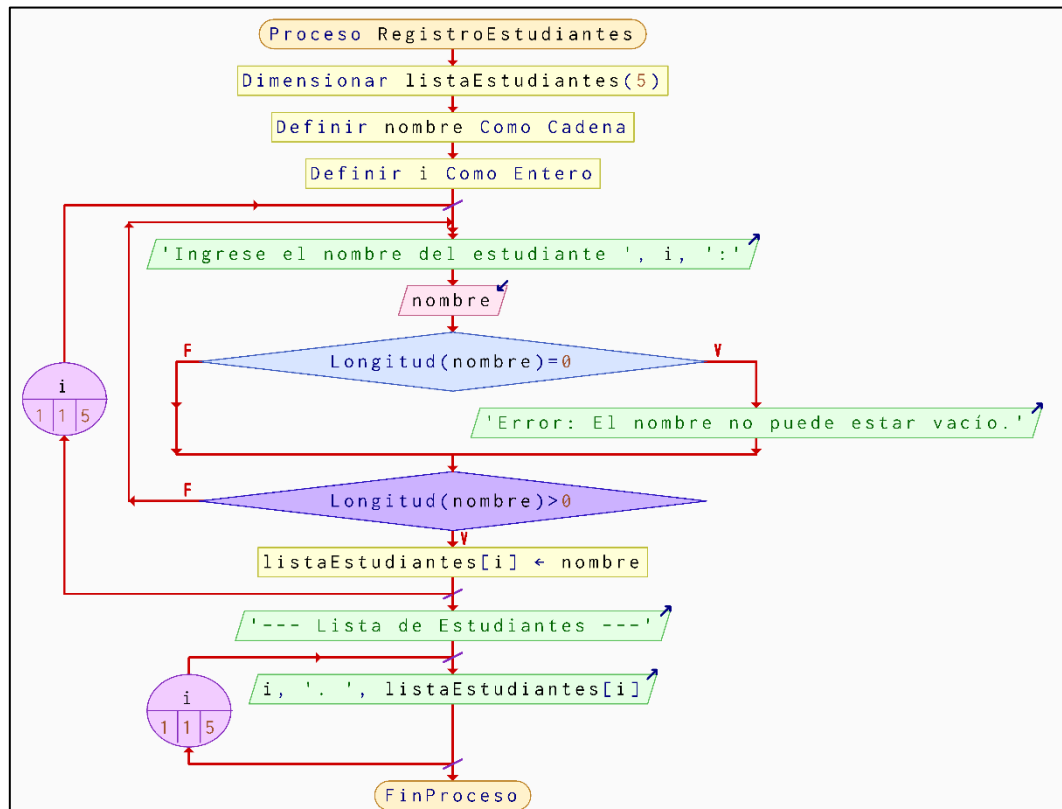
 listaEstudiantes[i] <- nombre

FinPara



Escribir "--- Lista de Estudiantes ---"
Para i <- 1 Hasta 5 Con Paso 1 Hacer
 Escribir i, ". ", listaEstudiantes[i]
FinPara
FinAlgoritmo

5. DIAGRAMA DE FLUJO



6. CÓDIGO EN C++

```
// C++ - Ejercicio 1
#include <iostream>
#include <vector> //
#include <string>

using namespace std;

int main() {
    vector<string> listaEstudiantes; //
    string nombre;

    for (int i = 1; i <= 5; i++) {
        do {
            cout << "Ingrese el nombre del estudiante " << i << ": ";
            getline(cin, nombre);
            if (nombre.empty() || nombre.find_first_not_of(' ') == string::npos) {
                cout << "Error: El nombre no puede estar vacío.\n";
            }
        } while (true);
        listaEstudiantes.push_back(nombre);
    }

    cout << "--- Lista de Estudiantes ---\n";
    for (int i = 1; i <= 5; i++) {
        cout << i << ". " << listaEstudiantes[i-1] << "\n";
    }
}
```



```
        nombre = "";
    }
    while (nombre.empty());
    listaEstudiantes.push_back(nombre); //
}

cout << "\n--- Lista de Estudiantes ---\n";
for (size_t i = 0; i < listaEstudiantes.size(); i++) {
    cout << (i + 1) << ". " << listaEstudiantes[i] << "\n"; //
}
return 0;
}
```

7. COMPILACIÓN EN C++

```
Ingrese el nombre del estudiante 1: Isaac Sanchez
Ingrese el nombre del estudiante 2: Melany Garcia
Ingrese el nombre del estudiante 3: Mishell Vilca
Ingrese el nombre del estudiante 4: Vanessa Pucuji
Ingrese el nombre del estudiante 5: Estefania Chamorro

--- Lista de Estudiantes ---
1. Isaac Sanchez
2. Melany Garcia
3. Mishell Vilca
4. Vanessa Pucuji
5. Estefania Chamorro
```

8. CÓDIGO EN JAVA

```
// Java - Ejercicio 1
import java.util.ArrayList; //
import java.util.Scanner;

public class Ejercicio1 {
    public static void main(String[] args) {
        ArrayList<String> listaEstudiantes = new ArrayList<>(); //
        Scanner scanner = new Scanner(System.in);

        for (int i = 1; i <= 5; i++) {
            String nombre = "";
            while (nombre.trim().isEmpty()) {
                System.out.print("Ingrese el nombre del estudiante " + i + ": ");
                nombre = scanner.nextLine();
                if (nombre.trim().isEmpty()) {
                    System.out.println("Error: El nombre no puede estar vacío.");
                }
            }
            listaEstudiantes.add(nombre); //
        }
    }
}
```



```
}  
  
System.out.println("\n--- Lista de Estudiantes ---");  
for (int i = 0; i < listaEstudiantes.size(); i++) {  
    System.out.println((i + 1) + ". " + listaEstudiantes.get(i)); //  
}  
}  
}
```

9. COMPILACIÓN EN JAVA

```
Ingrese el nombre del estudiante 1: Isaac Sanchez  
Ingrese el nombre del estudiante 2: Melany Sarahi  
Ingrese el nombre del estudiante 3: Alex Saltos  
Ingrese el nombre del estudiante 4: Gerardo Salazar  
Ingrese el nombre del estudiante 5: Cris Valencia
```

```
--- Lista de Estudiantes ---
```

```
1. Isaac Sanchez  
2. Melany Sarahi  
3. Alex Saltos  
4. Gerardo Salazar  
5. Cris Valencia
```

```
-----  
BUILD SUCCESS
```




Ejercicio 2: ArrayList I - Gestión de Notas

Enunciado: Crear un ArrayList con 6 notas.

1. ANÁLISIS DEL PROBLEMA

- Entrada: 6 notas numéricas (de tipo punto flotante).
- Proceso: Almacenar las notas en la lista dinámica. Validar estrictamente que cada nota esté dentro del rango académico estándar (0.0 a 10.0).
- Salida: Desplegar el listado de notas junto con el promedio final calculado.

2. DEFINICIÓN DE VARIABLES Y EQUIVALENCIAS

Componente del Programa	Tipo de Dato (Java)	Tipo de Dato (C++)	Descripción
listaNotas	ArrayList<Double>	vector<double>	Almacena las 6 notas de manera dinámica.
nota	double	double	Almacena el valor de la nota ingresada en el ciclo.
suma	double	double	Acumulador para obtener el promedio.

3. ALGORITMO LÓGICO

- Inicializar la colección dinámica de notas y el acumulador $suma = 0.0$.
- Ejecutar un bucle de 6 iteraciones:
- Solicitar una nota numérica.
- Validar que la nota pertenezca al rango $[0,10]$. Si no es así, lanzar mensaje de error y repetir lectura.
- Añadir la nota válida al contenedor.
- Sumar la nota leída al acumulador.
- Imprimir el set total de notas guardadas.
- Calcular y mostrar el promedio final dividiendo $suma/6$.

4. CÓDIGO EN PSEINT

Algoritmo GestionNotas

Dimension listaNotas[6]

Definir nota, suma, promedio Como Real

Definir i Como Entero

suma <- 0

Para i <- 1 Hasta 6 Con Paso 1 Hacer

Repetir

Escribir "Ingrese la nota ", i, " (0 - 10):"

Leer nota

Si nota < 0 O nota > 10 Entonces

Escribir "Error: La nota debe estar entre 0 y 10."

FinSi

Hasta Que nota >= 0 Y nota <= 10

listaNotas[i] <- nota

suma <- suma + nota



FinPara

promedio <- suma / 6

Escribir "--- Notas Registradas ---"

Para i <- 1 Hasta 6 Con Paso 1 Hacer

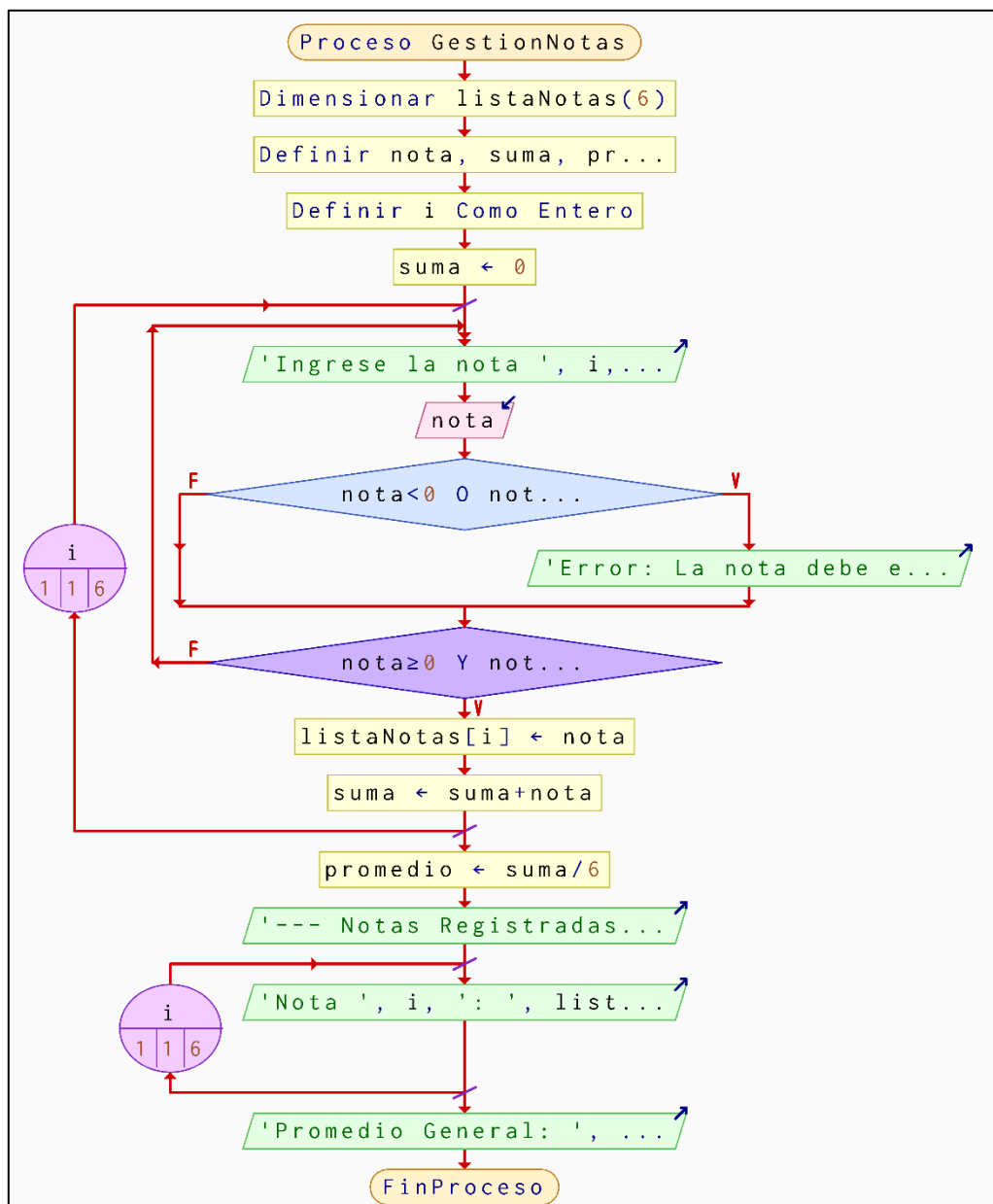
 Escribir "Nota ", i, ": ", listaNotas[i]

FinPara

Escribir "Promedio General: ", promedio

FinAlgoritmo

5. DIAGRAMA DE FLUJO



6. CÓDIGO EN C++

```
// C++ - Ejercicio 2
#include <iostream>
```



```
#include <vector> //
#include <iomanip>

using namespace std;

int main() {
    vector<double> listaNotas; //
    double nota, suma = 0.0;

    for (int i = 1; i <= 6; i++) {
        while (true) {
            cout << "Ingrese la nota " << i << " (0.0 - 10.0): ";
            if (cin >> nota) {
                if (nota >= 0 && nota <= 10) {
                    break;
                }
                cout << "Error: La nota debe estar en el rango de 0 a 10.\n";
            } else {
                cout << "Error: Entrada no numerica.\n";
                cin.clear();
                cin.ignore(10000, '\n');
            }
        }
        listaNotas.push_back(nota); //
        suma += nota;
    }

    cout << "\n--- Reporte de Notas ---\n" << fixed << setprecision(2);
    for (size_t i = 0; i < listaNotas.size(); i++) {
        cout << "Nota #" << (i + 1) << ": " << listaNotas[i] << "\n"; //
    }
    cout << "Promedio Final: " << (suma / listaNotas.size()) << "\n";
    return 0;
}
```

7. COMPILACIÓN EN C++

```
Ingrese la nota 1 (0.0 - 10.0): 10
Ingrese la nota 2 (0.0 - 10.0): 8.8
Ingrese la nota 3 (0.0 - 10.0): 9.7
Ingrese la nota 4 (0.0 - 10.0): 2.3
Ingrese la nota 5 (0.0 - 10.0): 2
Ingrese la nota 6 (0.0 - 10.0): 1

--- Reporte de Notas ---
Nota #1: 10.00
Nota #2: 8.80
Nota #3: 9.70
Nota #4: 2.30
Nota #5: 2.00
Nota #6: 1.00
Promedio Final: 5.63
```



8. CÓDIGO EN JAVA

```
// Java - Ejercicio 2
import java.util.ArrayList; //
import java.util.Scanner;

public class Ejercicio2 {
    public static void main(String[] args) {
        ArrayList<Double> listaNotas = new ArrayList<>(); //
        Scanner scanner = new Scanner(System.in);
        double suma = 0.0;

        for (int i = 1; i <= 6; i++) {
            double nota = -1;
            while (nota < 0 || nota > 10) {
                System.out.print("Ingrese la nota " + i + " (0.0 - 10.0): ");
                if (scanner.hasNextDouble()) {
                    nota = scanner.nextDouble();
                    if (nota < 0 || nota > 10) {
                        System.out.println("Error: Rango inválido.");
                    }
                } else {
                    System.out.println("Error: Ingrese un valor numérico válido.");
                    scanner.next(); // Limpiar token inválido
                }
            }
            listaNotas.add(nota); //
            suma += nota;
        }

        System.out.println("\n--- Reporte de Notas ---");
        for (int i = 0; i < listaNotas.size(); i++) {
            System.out.println("Nota #" + (i + 1) + ": " + listaNotas.get(i)); //
        }
        System.out.printf("Promedio Final: %.2f\n", (suma / listaNotas.size()));
    }
}
```

9. COMPILACIÓN EN JAVA

```
Ingrese la nota 1 (0.0 - 10.0): 6,7
Ingrese la nota 2 (0.0 - 10.0): 8,9
Ingrese la nota 3 (0.0 - 10.0): 6,5
Ingrese la nota 4 (0.0 - 10.0): 4,09
Ingrese la nota 5 (0.0 - 10.0): 8,9
Ingrese la nota 6 (0.0 - 10.0): 10

--- Reporte de Notas ---
Nota #1: 6.7
Nota #2: 8.9
Nota #3: 6.5
Nota #4: 4.09
Nota #5: 8.9
Nota #6: 10.0
Promedio Final: 7,52
-----
BUILD SUCCESS
```



Ejercicio 3: ArrayList I - Búsqueda de Productos

Enunciado: Registrar productos y realizar búsquedas.

1. ANÁLISIS DEL PROBLEMA

- **Entrada:** Nombres de múltiples productos para registrar; posteriormente, un nombre de producto a buscar.
- **Proceso:** Almacenar los productos ingresados por el usuario. Una vez finalizado el registro, permitir la búsqueda lineal de un ítem y verificar de forma exacta si se encuentra en la base de datos dinámica.
- **Salida:** Confirmación de la existencia del producto y su posición de índice.

2. DEFINICIÓN DE VARIABLES Y EQUIVALENCIAS

Componente del Programa	Tipo de Dato (Java)	Tipo de Dato (C++)	Descripción
inventario	ArrayList<String>	vector<string>	Vector dinámico contenedor de strings.
prodBuscar	String	string	Nombre del artículo a localizar dentro de la búsqueda.
encontrado	boolean	bool	Flag booleana que determina el éxito de la operación.

3. ALGORITMO LÓGICO

- Declarar la estructura dinámica de strings.
- Solicitar de forma secuencial los nombres de los productos al usuario hasta que decida terminar.
- Capturar la variable string prodBuscar.
- Evaluar mediante un bucle de comparación si prodBuscar coincide con algún elemento de la lista.
- Si coincide, cambiar encontrado a verdadero y guardar índice.
- Si finaliza el bucle y encontrado es falso, mostrar un mensaje informando que el producto no existe.

4. CÓDIGO EN PSEINT

Algoritmo BusquedaProductos

Dimension inventario[100]

Definir inventario, prodBuscar, opcion Como Cadena

Definir cant, i Como Entero

Definir encontrado Como Logico

cant <- 0

Repetir

 cant <- cant + 1

 Escribir "Ingrese el nombre del producto ", cant, ":"

 Leer inventario[cant]

 Escribir "¿Desea agregar otro producto? (s/n)"

 Leer opcion

Hasta Que opcion = "n" O opcion = "N"



Escribir "Ingrese el producto que desea buscar:"

Leer prodBuscar

encontrado <- Falso

Para i <- 1 Hasta cant Con Paso 1 Hacer

 Si inventario[i] = prodBuscar Entonces

 Escribir "Producto encontrado en la posicion: ", i

 encontrado <- Verdadero

 FinSi

FinPara

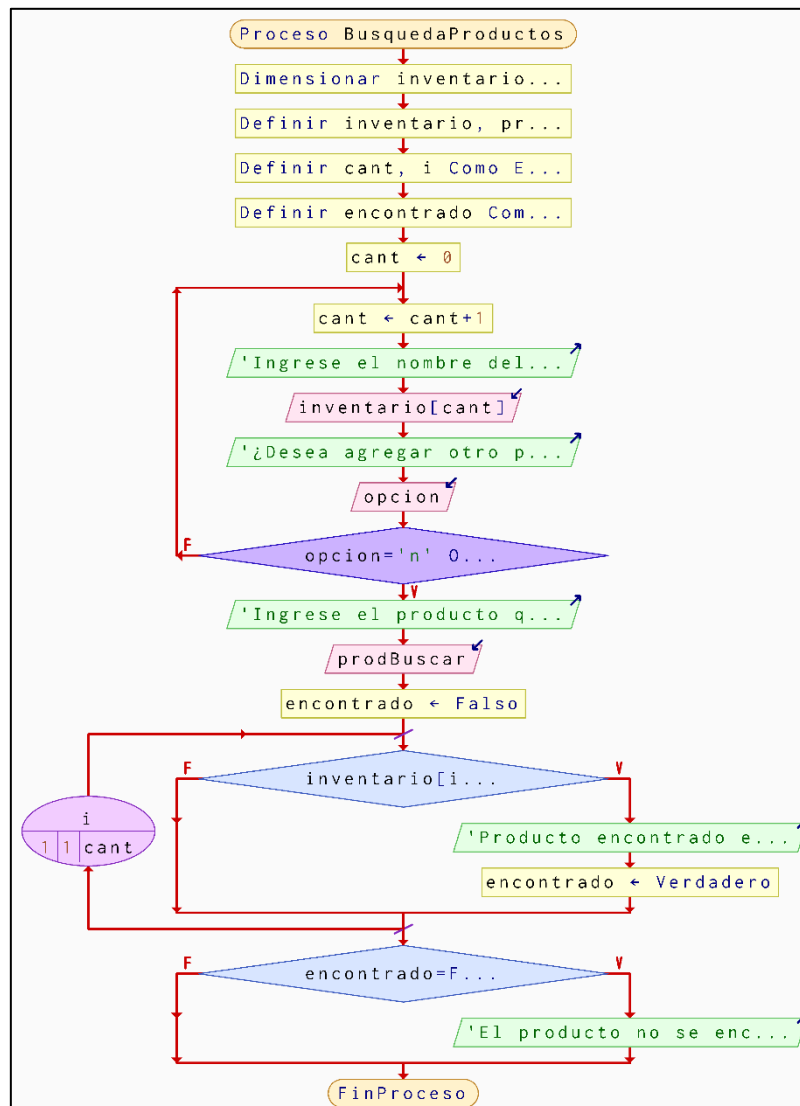
Si encontrado = Falso Entonces

 Escribir "El producto no se encuentra en el inventario."

FinSi

FinAlgoritmo

5. DIAGRAMA DE FLUJO





6. CÓDIGO EN C++

```
// C++ - Ejercicio 3
#include <iostream>
#include <vector> //
#include <string>
#include <algorithm>

using namespace std;

int main() {
    vector<string> inventario; //
    string prod, opcion, prodBuscar;

    do {
        cout << "Ingrese el nombre del producto a registrar: ";
        getline(cin, prod);
        if(!prod.empty()) {
            inventario.push_back(prod); //
        }
        cout << "¿Desea registrar otro? (s/n): ";
        getline(cin, opcion);
    } while(opcion == "s" || opcion == "S");

    cout << "\nIngrese el producto a buscar: ";
    getline(cin, prodBuscar);

    auto it = find(inventario.begin(), inventario.end(), prodBuscar);
    if(it != inventario.end()) {
        cout << "¡Producto encontrado! En la posicion: " << distance(inventario.begin(), it) << "\n";
    } else {
        cout << "El producto '" << prodBuscar << "' no existe en el registro.\n";
    }
    return 0;
}
```

7. COMPILACIÓN EN C++

```
Ingrese el nombre del producto a registrar: Arroz
¿Desea registrar otro? (s/n): s
Ingrese el nombre del producto a registrar: Papas
¿Desea registrar otro? (s/n): s
Ingrese el nombre del producto a registrar: Huevos
¿Desea registrar otro? (s/n): s
Ingrese el nombre del producto a registrar: Carne
¿Desea registrar otro? (s/n): s
Ingrese el nombre del producto a registrar: Pollo
¿Desea registrar otro? (s/n): n

Ingrese el producto a buscar: Carne
¡Producto encontrado! En la posicion: 3
```



8. CÓDIGO EN JAVA

```
// Java - Ejercicio 3
import java.util.ArrayList; //
import java.util.Scanner;

public class Ejercicio3 {
    public static void main(String[] args) {
        ArrayList<String> inventario = new ArrayList<>(); //
        Scanner scanner = new Scanner(System.in);
        String opcion;

        do {
            System.out.print("Ingrese el nombre del producto a registrar: ");
            String prod = scanner.nextLine().trim();
            if (!prod.isEmpty()) {
                inventario.add(prod); //
            }
            System.out.print("¿Desea registrar otro? (s/n): ");
            opcion = scanner.nextLine();
        } while (opcion.equalsIgnoreCase("s"));

        System.out.print("\nIngrese el producto a buscar: ");
        String prodBuscar = scanner.nextLine().trim();

        int pos = inventario.indexOf(prodBuscar);
        if (pos != -1) {
            System.out.println("¡Producto encontrado! Registrado en el índice: " + pos);
        } else {
            System.out.println("El producto '" + prodBuscar + "' no existe en el registro.");
        }
    }
}
```

9. COMPILACIÓN EN JAVA

```
Ingrese el nombre del producto a registrar: Carne
❖Desea registrar otro? (s/n): s
Ingrese el nombre del producto a registrar: Huevos
❖Desea registrar otro? (s/n): s
Ingrese el nombre del producto a registrar: Arroz
❖Desea registrar otro? (s/n): n

Ingrese el producto a buscar: Papas
El producto 'Papas' no existe en el registro.
-----
BUILD SUCCESS
```




Ejercicio 4: ArrayList II - Menú CRUD Completo

Enunciado: Crear un menú CRUD usando ArrayList.

1. ANÁLISIS DEL PROBLEMA

- **Entrada:** Selección de operaciones mediante un menú interactivo: (1: Crear, 2: Leer, 3: Actualizar, 4: Eliminar, 5: Salir) e ingresos variables según la opción.
- **Proceso:** Administrar un listado dinámico de elementos. Validar índices de modificación y borrado para evitar desbordamiento de memoria o interrupciones anormales en tiempo de ejecución.
- **Salida:** Visualización de los estados actualizados del almacén de datos.

2. DEFINICIÓN DE VARIABLES Y EQUIVALENCIAS

Componente del Programa	Tipo de Dato (Java)	Tipo de Dato (C++)	Descripción
crudList	ArrayList<String>	vector<string>	Lista principal del CRUD.
opcion	int	int	Control del switch del menú de opciones.
indice	int	int	Posición del elemento a modificar o eliminar.

3. ALGORITMO LÓGICO

- Desplegar de forma iterativa el menú en pantalla.
- Leer la opción deseada y validar que sea un valor numérico entre 1 y 5.
- Ejecutar la estructura de control switch según corresponda:
- Caso 1 (Create): Leer cadena e insertarla.
- Caso 2 (Read): Verificar si la lista está vacía; si no lo está, iterar y mostrar sus elementos.
- Caso 3 (Update): Leer índice, validar que cumpla con $0 \leq \text{índice} < \text{tamaño de lista}$, y reemplazar el contenido.
- Caso 4 (Delete): Leer índice, comprobar límites válidos y remover el elemento de la estructura.
- Repetir el flujo hasta capturar la opción de salida (5).

4. CÓDIGO EN PSEINT

Algoritmo CRUD_Menu

Dimension datos[50]

Definir datos, item Como Cadena

Definir opc, total, idx, i Como Entero

total <- 0; opc <- 0

Mientras opc <> 5 Hacer

 Escribir "--- MENU CRUD ---"

 Escribir "1. Crear (Agregar)"

 Escribir "2. Leer (Listar)"

 Escribir "3. Actualizar"

 Escribir "4. Eliminar"

 Escribir "5. Salir"

 Leer opc

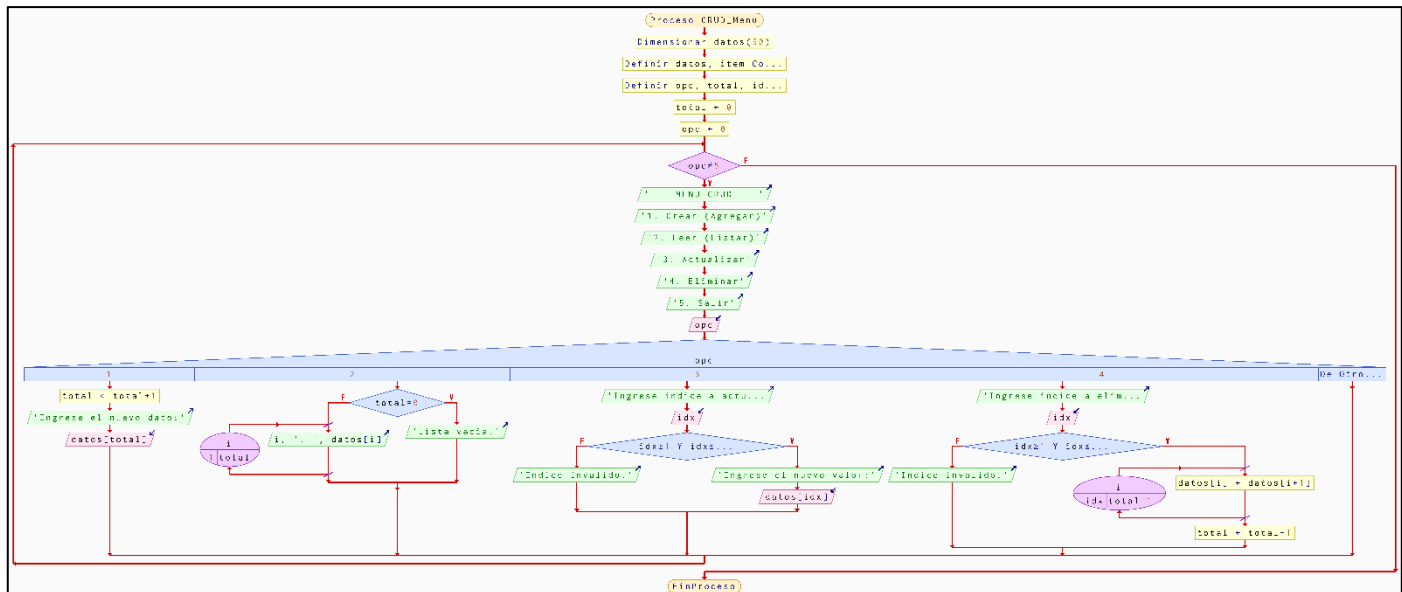


Segun opc Hacer

```
1:
  total <- total + 1
  Escribir "Ingrese el nuevo dato:"
  Leer datos[total]
2:
  Si total = 0 Entonces
    Escribir "Lista vacia."
  Sino
    Para i <- 1 Hasta total Hacer
      Escribir i, ". ", datos[i]
    FinPara
  FinSi
3:
  Escribir "Ingrese indice a actualizar:"
  Leer idx
  Si idx >= 1 Y idx <= total Entonces
    Escribir "Ingrese el nuevo valor:"
    Leer datos[idx]
  Sino
    Escribir "Indice invalido."
  FinSi
4:
  Escribir "Ingrese indice a eliminar:"
  Leer idx
  Si idx >= 1 Y idx <= total Entonces
    Para i <- idx Hasta total - 1 Hacer
      datos[i] <- datos[i+1]
    FinPara
    total <- total - 1
  Sino
    Escribir "Indice invalido."
  FinSi
FinSegun
FinMientras
FinAlgoritmo
```



5. DIAGRAMA DE FLUJO



6. CÓDIGO EN C++

```
// C++ - Ejercicio 4
#include <iostream>
#include <vector> //
#include <string>

using namespace std;

int main() {
    vector<string> crudList; //
    int opcion = 0;

    while (opcion != 5) {
        cout << "\n--- MODULO CRUD (C++) ---\n1. Registrar\n2. Listar\n3. Actualizar\n4.
Eliminar\n5. Salir\nOpcion: ";
        if (!(cin >> opcion)) {
            cout << "Entrada invalida.\n";
            cin.clear(); cin.ignore(10000, '\n');
            continue;
        }
        cin.ignore();

        if (opcion == 1) {
            string val; cout << "Contenido: "; getline(cin, val);
            crudList.push_back(val); //
        } else if (opcion == 2) {
            if (crudList.empty()) cout << "Vacio.\n";
            for (size_t i = 0; i < crudList.size(); ++i) cout << "[" << i << "]" << crudList[i] << "\n";
            //
        } else if (opcion == 3) {
            int idx; cout << "Indice: "; cin >> idx; cin.ignore();
            if (idx >= 0 && idx < (int)crudList.size()) {
                cout << "Nuevo valor: "; getline(cin, crudList[idx]);
            }
        } else if (opcion == 4) {
            int idx; cout << "Indice: "; cin >> idx; cin.ignore();
            if (idx >= 0 && idx < (int)crudList.size()) {
                crudList.erase(crudList.begin() + idx);
            }
        } else if (opcion == 5) {
            break;
        }
    }
}
```



```
        } else cout << "Fuera de rango.\n";
    } else if (opcion == 4) {
        int idx; cout << "Indice a eliminar: "; cin >> idx; cin.ignore();
        if (idx >= 0 && idx < (int)crudList.size()) {
            crudList.erase(crudList.begin() + idx); //
            cout << "Eliminado.\n";
        } else cout << "Fuera de rango.\n";
    }
}
return 0;
}
```

7. COMPILACIÓN EN C++

```
--- MODULO CRUD (C++) ---
1. Registrar
2. Listar
3. Actualizar
4. Eliminar
5. Salir
Opcion: 3
Indice: 0
Fuera de rango.

--- MODULO CRUD (C++) ---
1. Registrar
2. Listar
3. Actualizar
4. Eliminar
5. Salir
Opcion: 1
Contenido: Lenguaje
```

8. CÓDIGO EN JAVA

```
// Java - Ejercicio 4
import java.util.ArrayList; //
import java.util.Scanner;

public class Ejercicio4 {
    public static void main(String[] args) {
        ArrayList<String> crudList = new ArrayList<>(); //
        Scanner sc = new Scanner(System.in);
        int opcion = 0;

        while (opcion != 5) {
            System.out.println("\n--- MODULO CRUD (SISTEMA DE CONTROL) ---");
```



System.out.println("1. Registrar Elemento\n2. Listar Elementos\n3. Actualizar Elemento\n4. Eliminar Elemento\n5. Salir");
System.out.print("Seleccione opcion: ");

```
if (sc.hasNextInt()) {  
    opcion = sc.nextInt();  
    sc.nextLine(); // Limpiar buffer  
} else {  
    System.out.println("Opcion incorrecta.");  
    sc.nextLine();  
    continue;  
}  
  
switch (opcion) {  
    case 1 -> {  
        System.out.print("Contenido: ");  
        crudList.add(sc.nextLine()); //  
    }  
    case 2 -> {  
        if (crudList.isEmpty()) System.out.println("Coleccion vacia.");  
        else {  
            for (int i = 0; i < crudList.size(); i++) {  
                System.out.println "[" + i + " ] " + crudList.get(i); //  
            }  
        }  
    }  
    case 3 -> {  
        System.out.print("Indice a modificar: ");  
        int idx = sc.nextInt(); sc.nextLine();  
        if (idx >= 0 && idx < crudList.size()) {  
            System.out.print("Nuevo valor: ");  
            crudList.set(idx, sc.nextLine());  
        } else System.out.println("Indice fuera de rango.");  
    }  
    case 4 -> {  
        System.out.print("Indice a eliminar: ");  
        int delIdx = sc.nextInt(); sc.nextLine();  
        if (delIdx >= 0 && delIdx < crudList.size()) {  
            crudList.remove(delIdx); //  
            System.out.println("Eliminado correctamente.");  
        } else System.out.println("Indice fuera de rango.");  
    }  
}
```



9. COMPILACIÓN EN JAVA

```
--- MODULO CRUD (SISTEMA DE CONTROL) ---
1. Registrar Elemento
2. Listar Elementos
3. Actualizar Elemento
4. Eliminar Elemento
5. Salir
Seleccione opcion: 1
Contenido: 123

--- MODULO CRUD (SISTEMA DE CONTROL) ---
1. Registrar Elemento
2. Listar Elementos
3. Actualizar Elemento
4. Eliminar Elemento
5. Salir
Seleccione opcion: 1
Contenido: 124

--- MODULO CRUD (SISTEMA DE CONTROL) ---
1. Registrar Elemento
2. Listar Elementos
3. Actualizar Elemento
4. Eliminar Elemento
5. Salir
Seleccione opcion: 2
[0] 123
[1] 124
```



Ejercicio 5: ArrayList II - Registro Combinado de Alumnos y Calificaciones

Enunciado: Registrar nombres y notas.

1. ANÁLISIS DEL PROBLEMA

- **Entrada:** Nombres correspondientes a alumnos y sus respectivas notas de forma paralela.
- **Proceso:** Sincronizar dos estructuras dinámicas mediante punteros posicionales uniformes o índices idénticos. Validar de forma exhaustiva las entradas numéricas académicas de las notas.
- **Salida:** Una tabla estructurada que relacione a cada alumno con su respectiva calificación.

2. DEFINICIÓN DE VARIABLES Y EQUIVALENCIAS

Componente del Programa	Tipo de Dato (Java)	Tipo de Dato (C++)	Descripción
nombres	ArrayList<String>	vector<string>	Almacena los strings de los nombres.
notas	ArrayList<Double>	vector<double>	Almacena los doubles de las notas.
n	int	int	Cantidad total de registros que se ingresarán.

3. ALGORITMO LÓGICO

- Inicializar las dos colecciones paralelas (nombres y notas).
- Leer el número n de registros que se van a capturar.
- Desde $i = 1$ hasta n :
- Capturar el string de identificación del estudiante (validar que no sea vacío).
- Capturar la nota numérica y validar que cumpla el rango $[0,10]$.
- Insertar cada dato en sus listas correspondientes guardando simetría posicional.
- Imprimir la lista final recorriendo el índice en un único ciclo general.

4. CÓDIGO EN PSEINT

```
Algoritmo NombresNotasParalelos
Dimension arrayNombres[100], arrayNotas[100]
Definir arrayNombres, nom Como Cadena
Definir arrayNotas, nt Como Real
Definir n, i Como Entero

Escribir "Cantidad de registros a ingresar:"
Leer n

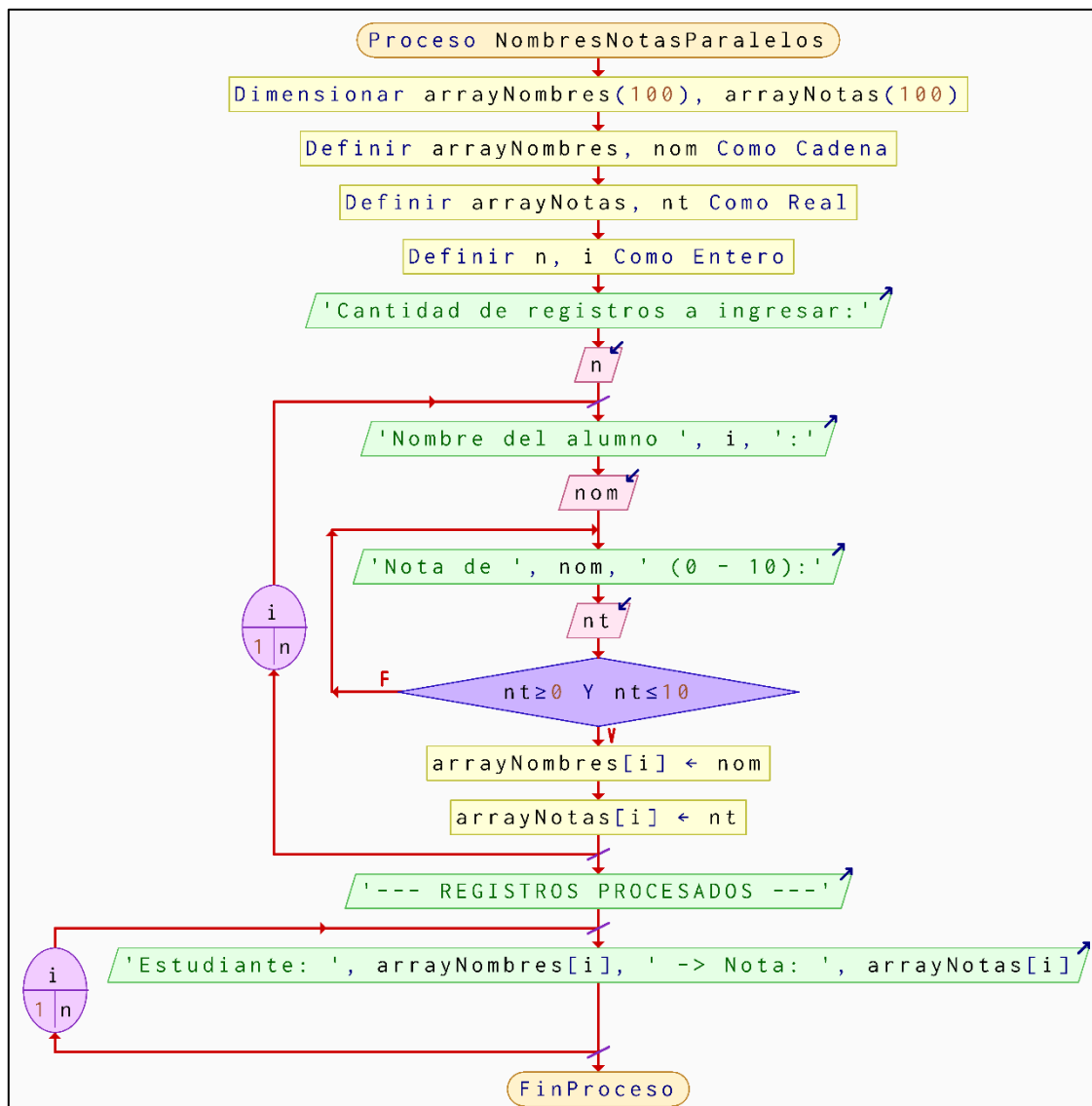
Para i <- 1 Hasta n Hacer
    Escribir "Nombre del alumno ", i, ":"
    Leer nom
    Repetir
        Escribir "Nota de ", nom, " (0 - 10):"
        Leer nt
    Hasta Que nt >= 0 Y nt <= 10
```



```
arrayNombres[i] <- nom  
arrayNotas[i] <- nt  
FinPara
```

```
Escribir "--- REGISTROS PROCESADOS ---"  
Para i <- 1 Hasta n Hacer  
    Escribir "Estudiante: ", arrayNombres[i], " -> Nota: ", arrayNotas[i]  
FinPara  
FinAlgoritmo
```

5. DIAGRAMA DE FLUJO



6. CÓDIGO EN C++

```
// C++ - Ejercicio 5  
#include <iostream>  
#include <vector>  
#include <string>
```




using namespace std;

```
int main() {
    vector<string> nombres; //
    vector<double> notas; //
    int n;

    cout << "Cantidad de estudiantes: ";
    cin >> n; cin.ignore();

    for (int i = 0; i < n; i++) {
        string nom; double nt = -1;
        cout << "Nombre del estudiante " << (i+1) << ": ";
        getline(cin, nom);
        nombres.push_back(nom); //

        while (nt < 0 || nt > 10) {
            cout << "Calificacion: ";
            if (!(cin >> nt)) { cin.clear(); cin.ignore(10000, '\n'); }
        }
        cin.ignore();
        notas.push_back(nt); //
    }

    cout << "\n--- BOLETIN ACADEMICO ---\n";
    for (size_t i = 0; i < nombres.size(); i++) {
        cout << "Alumno: " << nombres[i] << " \t| Nota: " << notas[i] << "\n"; //
    }
    return 0;
}
```

7. COMPILACIÓN EN C++

```
Cantidad de estudiantes: 3
Nombre del estudiante 1: Isaac
Calificacion: 10
Nombre del estudiante 2: Alex
Calificacion: 4
Nombre del estudiante 3: Alexa
Calificacion: 5

--- BOLETIN ACADEMICO ---
Alumno: Isaac    | Nota: 10
Alumno: Alex     | Nota: 4
Alumno: Alexa    | Nota: 5
```

8. CÓDIGO EN JAVA

// Java - Ejercicio 5



```
import java.util.ArrayList; //
import java.util.Scanner;

public class Ejercicio5 {
    public static void main(String[] args) {
        ArrayList<String> nombres = new ArrayList<>(); //
        ArrayList<Double> BlacklistNotas = new ArrayList<>(); //
        Scanner scanner = new Scanner(System.in);

        System.out.print("¿Cuántos estudiantes registrará?: ");
        int n = scanner.nextInt();
        scanner.nextLine();

        for (int i = 0; i < n; i++) {
            System.out.print("Nombre completo: ");
            nombres.add(scanner.nextLine()); //

            double nt = -1;
            while (nt < 0 || nt > 10) {
                System.out.print("Nota (0.0 - 10.0): ");
                if (scanner.hasNextDouble()) {
                    nt = scanner.nextDouble();
                } else scanner.next();
            }
            scanner.nextLine();
            BlacklistNotas.add(nt); //
        }

        System.out.println("\n--- BOLETÍN ACADÉMICO ---");
        for (int i = 0; i < nombres.size(); i++) {
            System.out.println("Alumno: " + nombres.get(i) + " | Calificación: " +
            BlacklistNotas.get(i)); //
        }
    }
}
```

9. COMPILACIÓN EN JAVA

```
Cuantos estudiantes registrara?: 3
Nombre completo: Alex
Nota (0.0 - 10.0): 10
Nombre completo: Isaac
Nota (0.0 - 10.0): 4,5
Nombre completo: Melany
Nota (0.0 - 10.0): 4

--- BOLETIN ACADEMICO ---
Alumno: Alex | Calificacion: 10.0
Alumno: Isaac | Calificacion: 4.5
Alumno: Melany | Calificacion: 4.0
-----
BUILD SUCCESS
-----
```



Ejercicio 6: ArrayList II - Sistema de Inventario

Enunciado: Crear un sistema de inventario.

1. ANÁLISIS DEL PROBLEMA

- **Entrada:** Altas de productos con código identificador, descripción de stock y precio unitario.
- **Proceso:** Modelar la entidad del producto y gestionarla mediante un arreglo dinámico de objetos o estructuras estructuradas. Controlar la consistencia de tipos numéricos no negativos.
- **Salida:** Visualización consolidada con los cálculos del valor monetario total en inventario.

2. DEFINICIÓN DE VARIABLES Y EQUIVALENCIAS

Componente del Programa	Tipo de Dato (Java)	Tipo de Dato (C++)	Descripción
inventario	ArrayList<Producto>	vector<Producto>	Lista dinámica orientada a objetos.
cantidad	int	int	Existencias físicas del producto (Stock).
precio	double	double	Costo unitario del ítem de inventario.

3. ALGORITMO LÓGICO

- Definir una estructura u objeto Producto que agrupe: nombre, stock y precio.
- Inicializar la lista dinámica contenedora de dicha clase.
- Capturar los registros del usuario validando rigurosamente que el stock y el precio sean ≥ 0 .
- Al listar, calcular para cada registro el valor de la expresión:
$$\text{Subtotal} = \text{cantidad} \times \text{precio}$$
- Acumular los subtotales para determinar y mostrar el valor de la inversión total del inventario.

4. CÓDIGO EN PSEINT

Algoritmo InventarioBasico

Dimension nombres[50], stocks[50], precios[50]

Definir nombres Como Cadena

Definir stocks, totalProd, i Como Entero

Definir precios, valorTotal, subtotal Como Real

totalProd <- 2 // Configurado estático para demostración

Para i <- 1 Hasta totalProd Hacer

 Escribir "Nombre del Producto:"

 Leer nombres[i]

 Escribir "Cantidad disponible:"

 Leer stocks[i]

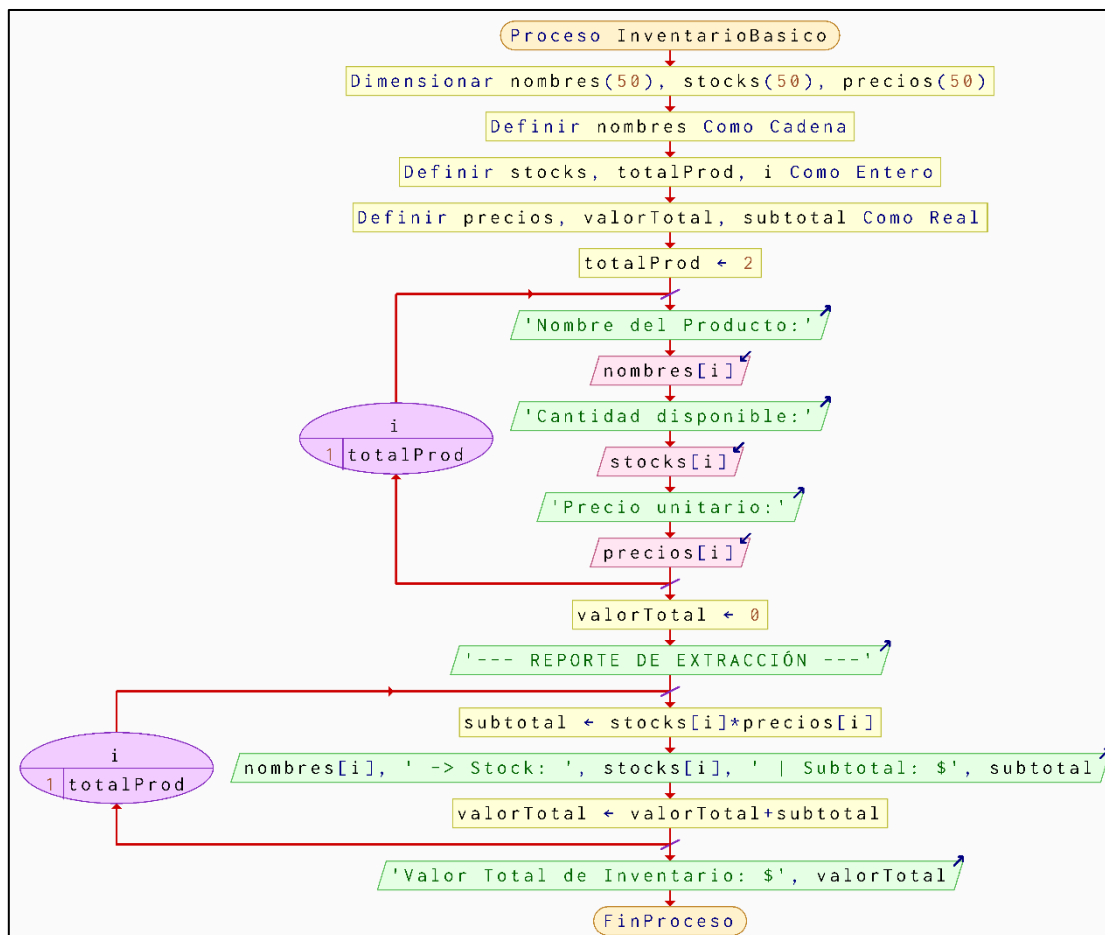
 Escribir "Precio unitario:"



```
Leer precios[i]
FinPara

valorTotal <- 0
Escribir "--- REPORTE DE EXTRACCIÓN ---"
Para i <- 1 Hasta totalProd Hacer
    subtotal <- stocks[i] * precios[i]
    Escribir nombres[i], " -> Stock: ", stocks[i], " | Subtotal: $", subtotal
    valorTotal <- valorTotal + subtotal
FinPara
Escribir "Valor Total de Inventario: $", valorTotal
FinAlgoritmo
```

5. DIAGRAMA DE FLUJO



6. CÓDIGO EN C++

```
// C++ - Ejercicio 6
#include <iostream>
#include <vector> //
#include <string>
```

```
using namespace std;
```



```
struct Producto {
    string nombre; int stock; double precio;
};

int main() {
    vector<Producto> inventario; //
    int n;
    cout << "Cantidad de productos: "; cin >> n;

    for (int i = 0; i < n; ++i) {
        cin.ignore(); Producto p;
        cout << "Nombre del Producto: "; getline(cin, p.nombre);
        p.stock = -1; p.precio = -1;
        while(p.stock < 0) { cout << "Stock: "; cin >> p.stock; }
        while(p.precio < 0) { cout << "Precio: "; cin >> p.precio; }
        inventario.push_back(p); //
    }

    double granTotal = 0;
    cout << "\n--- REPORTE DE VALORACION ---\n";
    for (const auto& prod : inventario) {
        double sub = prod.stock * prod.precio;
        cout << prod.nombre << " | Unidades: " << prod.stock << " | Subtotal: $" << sub << "\n";
        granTotal += sub;
    }
    cout << "Gran Total de Activos: $" << granTotal << "\n";
    return 0;
}
```

7. COMPILACIÓN EN C++

```
Cantidad de productos: 3
Nombre del Producto: Arroz
Stock: 3
Precio: 10.3
Nombre del Producto: Papas
Stock: 10
Precio: 10.9
Nombre del Producto: Carne
Stock: 19293
Precio: 1.4

--- REPORTE DE VALORACION ---
Arroz | Unidades: 3 | Subtotal: $30.9
Papas | Unidades: 10 | Subtotal: $109
Carne | Unidades: 19293 | Subtotal: $27010.2
Gran Total de Activos: $27150.1
```



8. CÓDIGO EN JAVA

```
// Java - Ejercicio 6
import java.util.ArrayList; //
import java.util.Scanner;

class Producto {
    String nombre; int stock; double precio;
    public Producto(String n, int s, double p) {
        this.nombre = n; this.stock = s; this.precio = p;
    }
}

public class Ejercicio6 {
    public static void main(String[] args) {
        ArrayList<Producto> inventario = new ArrayList<>(); //
        Scanner sc = new Scanner(System.in);

        System.out.print("Cantidad de tipos de productos a ingresar: ");
        int n = sc.nextInt(); sc.nextLine();

        for (int i = 0; i < n; i++) {
            System.out.print("Nombre del Producto: "); String nom = sc.nextLine();
            int stk = -1; double prc = -1;
            while(stk < 0) { System.out.print("Stock: "); stk = sc.nextInt(); }
            while(prc < 0) { System.out.print("Precio: "); prc = sc.nextDouble(); }
            sc.nextLine();
            inventario.add(new Producto(nom, stk, prc)); //
        }

        double granTotal = 0;
        System.out.println("\n--- VALORACIÓN TOTAL ---");
        for (Producto p : inventario) {
            double sub = p.stock * p.precio;
            System.out.println(p.nombre + " -> Cantidad: " + p.stock + " | Total: $" + sub);
            granTotal += sub;
        }
        System.out.println("Inversión de Activos Totales: $" + granTotal);
    }
}
```

9. COMPILACIÓN EN JAVA

```
Cantidad de tipos de productos a ingresar: 3
Nombre del Producto: Arroz
Stock: 10
Precio: 12,43
Nombre del Producto: Papas
Stock: 13
Precio: 12,34
Nombre del Producto: Carne
Stock: 123
Precio: 5,43

--- VALORACION TOTAL ---
Arroz -> Cantidad: 10 | Total: $124.3
Papas -> Cantidad: 13 | Total: $160.42
Carne -> Cantidad: 123 | Total: $667.89
Inversion de Activos Totales: $952.6099999999999
-----
BUILD SUCCESS
```



Ejercicio 7: Arreglos vs ArrayList - Estático vs Dinámico

Enunciado: Comparar arreglos y ArrayList.

1. ANÁLISIS DEL PROBLEMA

- **Concepto:** Demostrar en código la diferencia fundamental entre un arreglo clásico (tamaño fijo e inmutable en tiempo de ejecución) y un objeto dinámico (ArrayList/vector).
- **Proceso:** Crear ambas estructuras. Intentar poblar u operar sobre ellas para evidenciar la flexibilidad de redimensionamiento de las colecciones dinámicas frente a las estáticas.

2. DEFINICIÓN DE VARIABLES Y EQUIVALENCIAS

Componente del Programa	Tipo de Dato (Java)	Tipo de Dato (C++)	Descripción
arregloEstatico	int[]	int[3]	Arreglo de tamaño fijo inmutable.
listaDinamica	ArrayList<Integer>	vector<int>	Contenedor dinámico expansible.

3. ALGORITMO LÓGICO

- Declarar un arreglo nativo asignándole exactamente una dimensión fija de 3 posiciones.
- Instanciar una lista dinámica (ArrayList/vector).
- Llenar el arreglo estático hasta su límite de memoria física.
- Agregar elementos a la estructura dinámica superando las 3 posiciones iniciales para evidenciar que se redimensiona automáticamente de forma transparente al usuario.
- Imprimir la longitud final de ambos objetos para contrastarlos.

4. CÓDIGO EN PSEINT

Algoritmo EstaticoVsDinamico

// En PSeInt todas las estructuras se comportan como estáticas

Dimension estatico[3]

Escribir "Estructura fija inicializada a 3 posiciones."

estatico[1] <- 10

estatico[2] <- 20

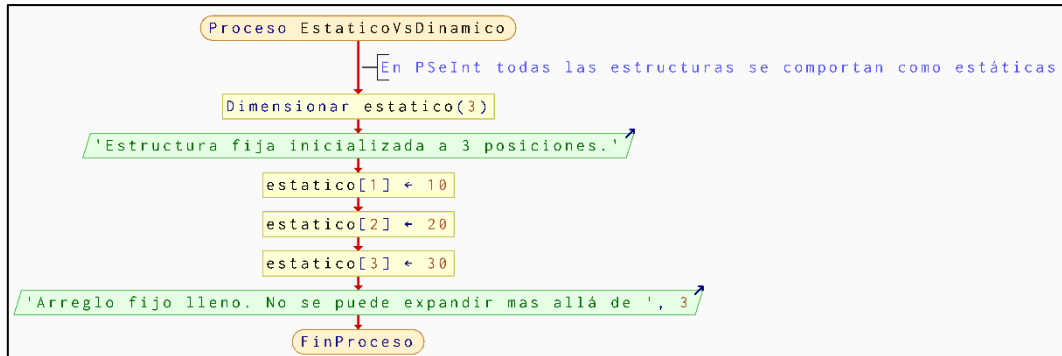
estatico[3] <- 30

Escribir "Arreglo fijo lleno. No se puede expandir mas allá de ", 3

FinAlgoritmo



5. DIAGRAMA DE FLUJO



6. CÓDIGO EN C++

```
// C++ - Ejercicio 7
#include <iostream>
#include <vector>

using namespace std;

int main() {
    // Estructura Estática
    int arregloEstatico[3] = {10, 20, 30};

    // Estructura Dinámica
    vector<int> listaDinamica; //
    listaDinamica.push_back(10); //
    listaDinamica.push_back(20); //
    listaDinamica.push_back(30); //
    listaDinamica.push_back(40); // Se expande de forma automatica en memoria

    cout << "Dimension del arreglo nativo fijo: " << 3 << "\n";
    cout << "Dimension actual de la lista dinamica: " << listaDinamica.size() << "\n";
    return 0;
}
```

7. COMPILACIÓN EN C++

```
Dimension del arreglo nativo fijo: 3
Dimension actual de la lista dinamica: 4

=== Code Execution Successful ===
```

8. CÓDIGO EN JAVA

```
// Java - Ejercicio 7
import java.util.ArrayList;

public class Ejercicio7 {
    public static void main(String[] args) {
```




```
// Estructura Estática (Tamaño Fijo)
int[] arregloEstatico = new int[3];
arregloEstatico[0] = 100;
arregloEstatico[1] = 200;
arregloEstatico[2] = 300;
// arregloEstatico[3] = 400; // Provocaría un ArrayIndexOutOfBoundsException

// Estructura Dinámica (Redimensionable)
ArrayList<Integer> listaDinamica = new ArrayList<>(); //
listaDinamica.add(100); //
listaDinamica.add(200); //
listaDinamica.add(300); //
listaDinamica.add(400); // Supera el tamaño inicial sin errores

System.out.println("Longitud inmutable del arreglo fijo: " + arregloEstatico.length);
System.out.println("Tamaño adaptado de la lista dinámica: " + listaDinamica.size());
}
}
```

9. COMPILACIÓN EN JAVA

```
--- exec:3.1.0:exec (default-cli) @ mavenproject1 ---
Longitud inmutable del arreglo fijo: 3
Tamano adaptado de la lista dinamica: 4
-----
BUILD SUCCESS
```



Ejercicio 8: Arreglos vs ArrayList - Rendimiento de Inserción

Enunciado: Comparar inserción de datos.

1. ANÁLISIS DEL PROBLEMA

- **Concepto:** Analizar la velocidad de ejecución y comportamiento al realizar operaciones masivas de adición entre bloques estáticos pre-assignados y listas que alteran dinámicamente sus punteros internos de memoria.
- **Proceso:** Ejecutar una iteración repetitiva de gran escala (ej. 10,000 inserciones) y medir la respuesta del sistema.

2. DEFINICIÓN DE VARIABLES Y EQUIVALENCIAS

Componente del Programa	Tipo de Dato (Java)	Tipo de Dato (C++)	Descripción
limite	final int	const int	Constante fijada en 10000 para las pruebas de esfuerzo.
vDinamico	ArrayList<Integer>	vector<int>	Contenedor dinámico para la prueba de velocidad.

3. ALGORITMO LÓGICO

- Definir una escala de iteración numérica moderada-alta (*Muestra: 10,000\$ ciclos*).
- Inicializar un cronómetro de sistema para registrar marcas de tiempo de alta precisión.
- Ejecutar la inserción masiva secuencial sobre la estructura dinámica.
- Detener el reloj y calcular la diferencia temporal absoluta.
- Imprimir el tiempo total empleado en milisegundos para evaluar la eficiencia de la estructura dinámicamente redimensionada.

4. CÓDIGO EN PSEINT

Algoritmo RendimientoInsercion

Definir iteraciones, i Como Entero

iteraciones <- 10000

Escribir "Iniciando simulacion de insercion masiva de ", iteraciones, " elementos..."

Para i <- 1 Hasta iteraciones Hacer

 // Simulación de paso operacional

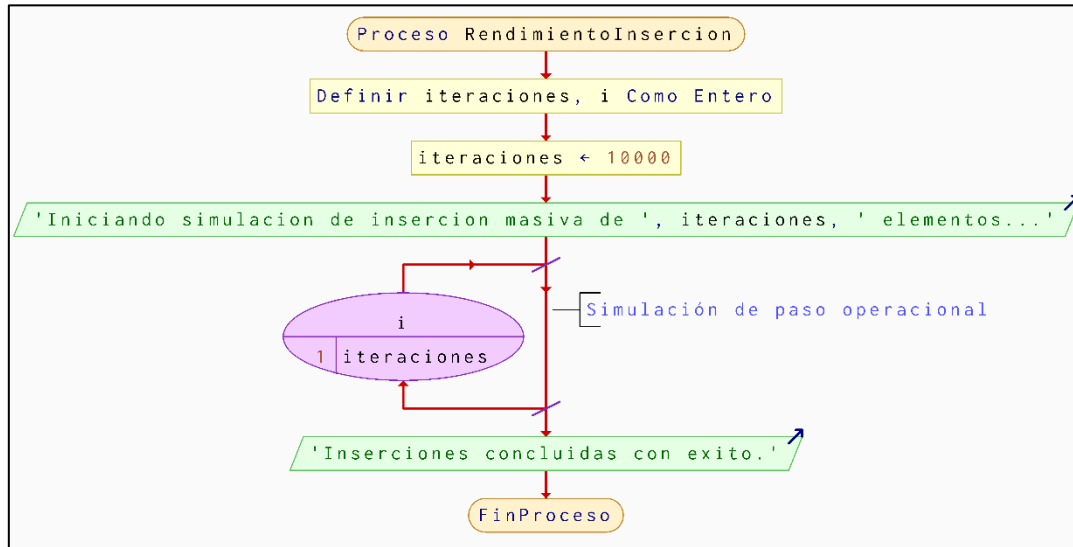
FinPara

Escribir "Inserciones concluidas con exito."

FinAlgoritmo



5. DIAGRAMA DE FLUJO



6. CÓDIGO EN C++

```
// C++ - Ejercicio 8
#include <iostream>
#include <vector> //
#include <chrono>

using namespace std;
using namespace std::chrono;

int main() {
    const int LIMITE = 10000;
    vector<int> vDinamico; //

    auto tInicio = high_resolution_clock::now();
    for (int i = 0; i < LIMITE; ++i) {
        vDinamico.push_back(i); //
    }
    auto tFin = high_resolution_clock::now();

    duration<double, milli> tTotal = tFin - tInicio;
    cout << "Insercion de " << LIMITE << " enteros en vector C++: " << tTotal.count() << " ms\n";
    return 0;
}
```

7. COMPILACIÓN EN C++

```
Insercion de 10000 enteros en vector C++: 0.16656 ms
```

```
=== Code Execution Successful ===
```



8. CÓDIGO EN JAVA

```
// Java - Ejercicio 8
import java.util.ArrayList; //

public class Ejercicio8 {
    public static void main(String[] args) {
        final int LIMITE = 10000;
        ArrayList<Integer> vDinamico = new ArrayList<>(); //

        long tInicio = System.nanoTime();
        for (int i = 0; i < LIMITE; i++) {
            vDinamico.add(i); //
        }
        long tFin = System.nanoTime();

        double milisegundos = (tFin - tInicio) / 1000000.0;
        System.out.println("Insercion de " + LIMITE + " enteros en ArrayList: " + milisegundos + "
ms");
    }
}
```

9. COMPILACIÓN EN JAVA

```
--- exec:3.1.0:exec (default-cli) @ mavenproject1 ---
Insercion de 10000 enteros en ArrayList: 3.9947 ms
-----
BUILD SUCCESS
-----
```



Ejercicio 9: LinkedList - Asignación de Turnos

Enunciado: Manipular turnos con LinkedList.

1. Análisis del Problema

- **Entrada:** Nombres de personas que solicitan un turno de atención.
- **Proceso:** Insertar elementos en una lista doblemente enlazada (LinkedList/list). El uso de nodos enlazados permite optimizar las operaciones de inserción y eliminación en los extremos de la lista frente a un ArrayList convencional.
- **Salida:** Visualización ordenada de los turnos vigentes en el sistema.

2. DEFINICIÓN DE VARIABLES Y EQUIVALENCIAS

Componente del Programa	Tipo de Dato (Java)	Tipo de Dato (C++)	Descripción
colaTurnos	LinkedList<String>	list<string>	Estructura de lista enlazada para los turnos.
paciente	String	string	Nombre de la persona asignada al turno.

3. ALGORITMO LÓGICO

- Instanciar el objeto de lista enlazada (LinkedList / list).
- Leer y almacenar las identidades asociadas a los turnos utilizando inserciones al final de la estructura (add / push_back).
- Validar la consistencia de los datos de entrada para evitar nodos con valores nulos o vacíos.
- Recorrer secuencialmente los eslabones de la lista enlazada para imprimir los turnos en orden de llegada.

4. CÓDIGO EN PSEINT

Algoritmo AsignacionTurnos

Dimension colaTurnos[5]

Definir colaTurnos, cliente Como Cadena

Definir i Como Entero

Para i <- 1 Hasta 3 Hacer

 Escribir "Asigne nombre para el turno ", i, ":"

 Leer cliente

 colaTurnos[i] <- cliente

FinPara

Escribir "--- TURNOS DE ATENCIÓN EMITIDOS ---"

Para i <- 1 Hasta 3 Hacer

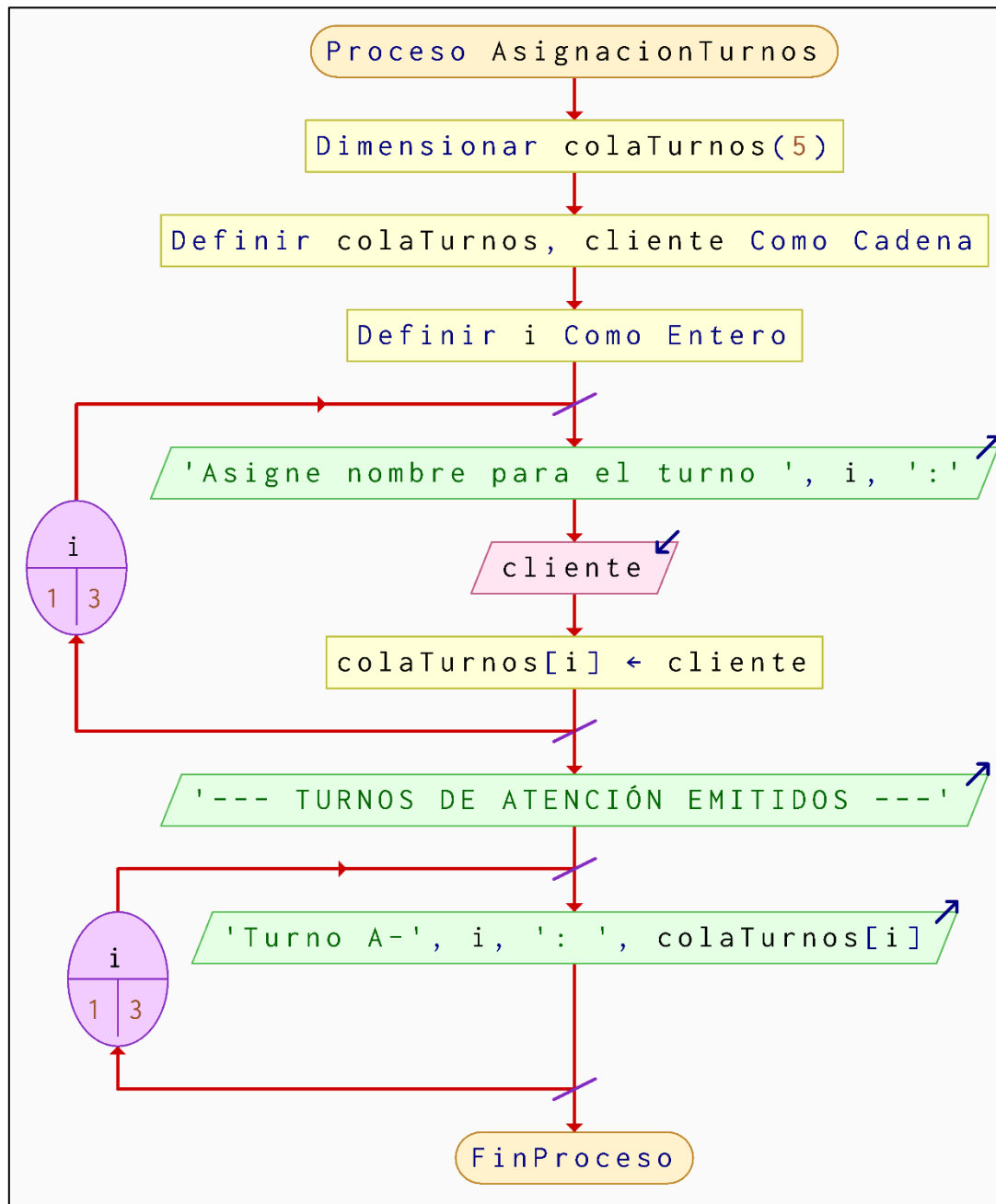
 Escribir "Turno A-", i, ": ", colaTurnos[i]

FinPara

FinAlgoritmo



5. DIAGRAMA DE FLUJO



6. CÓDIGO EN C++

```
// C++ - Ejercicio 9
#include <iostream>
#include <list> //
#include <string>

using namespace std;

int main() {
    list<string> colaTurnos; //
```



string t;

```
cout << "--- REGISTRO DE TURNOS (C++ LIST) ---\n";
for (int i = 1; i <= 3; i++) {
    cout << "Nombre para el turno #" << i << ": ";
    getline(cin, t);
    if(!t.empty()) {
        colaTurnos.push_back(t); //
    } else {
        cout << "Entrada invalida.\n";
        i--;
    }
}

cout << "\n--- PANTALLA DE TURNOS ACTUALES ---\n";
int index = 1;
for (const string& turno : colaTurnos) {
    cout << "Codigo: T-" << index << " | Nombre: " << turno << "\n";
    index++;
}
return 0;
}
```

7. COMPILACIÓN EN C++

```
--- REGISTRO DE TURNOS (C++ LIST) ---
Nombre para el turno #1: Alex
Nombre para el turno #2: Isaac
Nombre para el turno #3: Melany

--- PANTALLA DE TURNOS ACTUALES ---
Codigo: T-1 | Nombre: Alex
Codigo: T-2 | Nombre: Isaac
Codigo: T-3 | Nombre: Melany
```

8. CÓDIGO EN JAVA

```
// Java - Ejercicio 9
import java.util.LinkedList; //
import java.util.Scanner;

public class Ejercicio9 {
    public static void main(String[] args) {
        LinkedList<String> colaTurnos = new LinkedList<>(); //
        Scanner scanner = new Scanner(System.in);

        System.out.println("--- REGISTRO DE TURNOS (LINKEDLIST) ---");
        for (int i = 1; i <= 3; i++) {
            System.out.print("Nombre para el turno #" + i + ": ");
```



```
String t = scanner.nextLine().trim();
if(!t.isEmpty()) {
    colaTurnos.add(t); //
} else {
    System.out.println("Entrada inválida. Intente de nuevo.");
    i--;
}
}

System.out.println("\n--- PANTALLA DE TURNOS ACTUALES ---");
int index = 1;
for (String turno : colaTurnos) {
    System.out.println("Código: T-" + index + " | Nombre: " + turno);
    index++;
}
}
```

9. COMPILACIÓN EN JAVA

```
--- REGISTRO DE TURNOS (LINKEDLIST) ---
Nombre para el turno #1: Adrian
Nombre para el turno #2: Isaac
Nombre para el turno #3: Melany

--- PANTALLA DE TURNOS ACTUALES ---
Codigo: T-1 | Nombre: Adrian
Codigo: T-2 | Nombre: Isaac
Codigo: T-3 | Nombre: Melany
-----
BUILD SUCCESS
```




Ejercicio 10: LinkedList - Simulación de una Fila de Atención (FIFO)

Enunciado: Simular una fila de atención.

1. ANÁLISIS DEL PROBLEMA

- **Concepto:** Simular una estructura de datos FIFO (First-In, First-Out) utilizando una lista enlazada. El primer elemento en ingresar a la fila debe ser el primero en ser atendido y removido del frente de la lista.
- **Entrada:** Nombres de clientes que ingresan a la fila de atención.
- **Proceso:** Encolar elementos al final de la estructura y desencolar desde el inicio, verificando que la lista contenga elementos antes de intentar remover un nodo.
- **Salida:** Registro de los clientes atendidos en orden cronológico y estado actual de la fila.

2. DEFINICIÓN DE VARIABLES Y EQUIVALENCIAS

Componente del Programa	Tipo de Dato (Java)	Tipo de Dato (C++)	Descripción
filaEspera	LinkedList<String>	list<string>	Lista enlazada que simula la fila de atención.
clienteAtendido	String	string	Almacena temporalmente el nombre del cliente extraído.

3. ALGORITMO LÓGICO

- Instanciar la lista enlazada vacía.
- Encolar tres clientes agregándolos al final de la lista de forma sucesiva (add / push_back).
- Mostrar el estado inicial de la fila de atención.
- Mientras la fila no esté vacía:
- Extraer el elemento que se encuentra al frente de la estructura.
- En Java se puede utilizar .removeFirst() y en C++ recuperar .front() y aplicar .pop_front().
- Imprimir en pantalla el nombre del cliente que está siendo atendido.
- Finalizar cuando no queden nodos en la lista enlazada.

4. CÓDIGO EN PSEINT

Algoritmo SimulacionFilaFIFO

Dimension fila[3]

Definir fila Como Cadena

Definir i Como Entero

fila[1] <- "Carlos"

fila[2] <- "Ana"

fila[3] <- "Mateo"

Escribir "--- PROCESANDO FILA DE ATENCIÓN (FIFO) ---"

Para i <- 1 Hasta 3 Hacer

 Escribir "Atendiendo al cliente: ", fila[i]

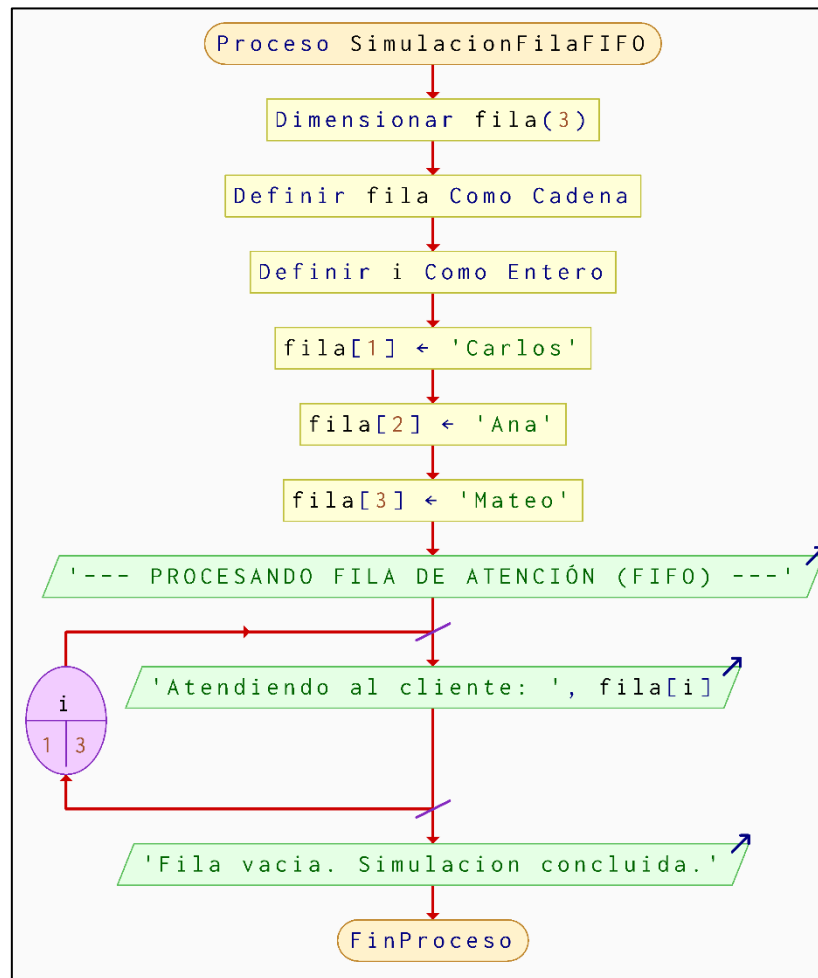
FinPara

Escribir "Fila vacia. Simulacion concluida."

FinAlgoritmo



5. DIAGRAMA DE FLUJO



6. CÓDIGO EN C++

```
// C++ - Ejercicio 10
#include <iostream>
#include <list> //
#include <string>

using namespace std;

int main() {
    list<string> filaEspera; //

    // Operación Encolar
    filaEspera.push_back("Carlos"); //
    filaEspera.push_back("Ana"); //
    filaEspera.push_back("Mateo"); //

    cout << "--- SIMULACION DE ATENCION DE CLIENTES (FIFO) ---\n\n";

    // Operación Desencolar
    while (!filaEspera.empty()) {
```



```
string clienteAtendido = filaEspera.front(); // Obtiene el frente de la fila
cout << "-> Atendiendo a: " << clienteAtendido << " ... Sesion Cerrada.\n";
filaEspera.pop_front(); // Remueve el primer nodo de la lista
}

cout << "\nEstado de Fila: Vacía. Todos los turnos del día fueron despachados.\n";
return 0;
}
```

7. COMPILACIÓN EN C++

```
--- SIMULACIÓN DE ATENCIÓN DE CLIENTES {FIFO} ---

-> Atendiendo a: Carlos ... Sesion Cerrada.
-> Atendiendo a: Ana ... Sesion Cerrada.
-> Atendiendo a: Mateo ... Sesion Cerrada.

Estado de Fila: Vacía. Todos los turnos del día fueron despachados.
```

8. CÓDIGO EN JAVA

```
// Java - Ejercicio 10
import java.util.LinkedList; //

public class Ejercicio10 {
    public static void main(String[] args) {
        LinkedList<String> filaEspera = new LinkedList<>(); //

        // Operación Encolar (Push back)
        filaEspera.add("Carlos"); //
        filaEspera.add("Ana"); //
        filaEspera.add("Mateo"); //

        System.out.println("Fila inicial de atención: " + filaEspera);
        System.out.println("\n--- INICIO DE ATENCIÓN DE CLIENTES ---");

        // Operación Desencolar (FIFO)
        while (!filaEspera.isEmpty()) {
            String clienteAtendido = filaEspera.removeFirst(); // Remueve el elemento del frente
            System.out.println("-> Atendiendo a: " + clienteAtendido + " ... Operación Completada.");
        }

        System.out.println("\nEstado Final: Todos los usuarios de la fila han sido atendidos.");
    }
}
```



9. COMPILACIÓN EN JAVA

```
Fila inicial de atencion: [Carlos, Ana, Mateo]

--- INICIO DE ATENCION DE CLIENTES ---
-> Atendiendo a: Carlos ... OperaciOn Completada.
-> Atendiendo a: Ana ... OperaciOn Completada.
-> Atendiendo a: Mateo ... OperaciOn Completada.

Estado Final: Todos los usuarios de la fila han sido atendidos.
-----
```